ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи №6

«Автоматизоване тестування Python-програм»

з дисципліни «Професійний Python»

Виконав: студент групи ФеП-33с

Цястун Павло

Варіант 1

Перевірив: професор Свелеба Сергій Андрійович

Львів – 2026

1. Мета роботи

Сформувати практичні навички професійного автоматизованого тестування Python-програм із використанням фреймворку pytest: проєктувати unit та integration tests, створювати fixtures, використовувати параметризацію, перевіряти exceptions, ізолювати файлову систему та зовнішні залежності, застосовувати mocks і аналізувати test coverage.

2. Номер варіанта та постановка задачі

Варіант 1. Система обліку студентів.

Необхідно створити повноцінний automated test suite для Python-проєкту варіанта 1. Для нього потрібно протестувати:

- створення Student та валідацію обов'язкових полів;
- пошук студента за прізвищем або ID;
- фільтрацію студентів за групою;
- обчислення середнього бала та визначення найкращого студента;
- порожню колекцію студентів;
- boundary та invalid значення середнього бала;
- CSV import студентів та malformed CSV record;
- JSON export;
- YAML/configuration parameters;
- StudentRepository через Mock або fake;
- Notifier через Mock та interaction assertions;
- failure path зовнішньої залежності через side_effect;
- async student service/repository через AsyncMock та integration flow import → validation → report.

3. Тестований проєкт

Для лабораторної роботи створено невеликий проєкт student_app (src-layout) на основі попередніх лабораторних: студентський реєстр із CSV-імпортом, YAML-конфігурацією, звітом, JSON-експортом та двома сервісами (синхронним і асинхронним). Залежності від зовнішнього світу (репозиторій і сповіщення) описано Protocol-інтерфейсами й передаються в сервіс ззовні, тому їх легко підміняти у тестах.

Модуль	Призначення
models.py	Student (frozen dataclass із валідацією), classify_grade (три гілки)
exceptions.py	ієрархія помилок: ValidationError (field), ImportDataError (line_number), ExportError, NotFoundError, NotificationError, ConfigurationError
protocols.py	StudentRepository, AsyncStudentRepository, Notifier
repositories.py	fake-репозиторії в пам’яті (синхронний і асинхронний)
services.py	чисті функції пошуку, фільтрації та статистики; StudentService (register, find_*, average_grade, best_student)
async_service.py	AsyncStudentService (load, average_grade, best_student)
importer.py	потоковий CSV-імпорт із валідацією, strict/skip_invalid
exporters.py	атомарний експорт у JSON
config.py	YAML-конфігурація, override змінною STUDENT_APP_MINIMUM_GRADE, get_environment
reports.py	build_report, utcnow (замінюється у тестах)
pipeline.py, main.py	імпорт → валідація → фільтр → звіт → експорт; CLI
notifiers.py	ConsoleNotifier

4. Структура test suite

```
student_project/
├── pyproject.toml          (pytest, coverage, markers)
├── config.yaml
├── data/students.csv
├── src/student_app/        (production code)
├── tests/
│   ├── conftest.py         (спільні fixtures)
│   ├── unit/
│   │   ├── test_models.py
│   │   ├── test_selectors.py
│   │   ├── test_services.py
│   │   ├── test_importer.py
│   │   ├── test_exporters.py
│   │   ├── test_configuration.py
│   │   ├── test_reports.py
│   │   ├── test_repositories.py
│   │   ├── test_notifiers.py
│   │   └── test_async_service.py
│   └── integration/
│       ├── test_import_pipeline.py
│       ├── test_export_pipeline.py
│       └── test_application_flow.py
└── experiments/            (експерименти 1, 2, 3, 5)
```

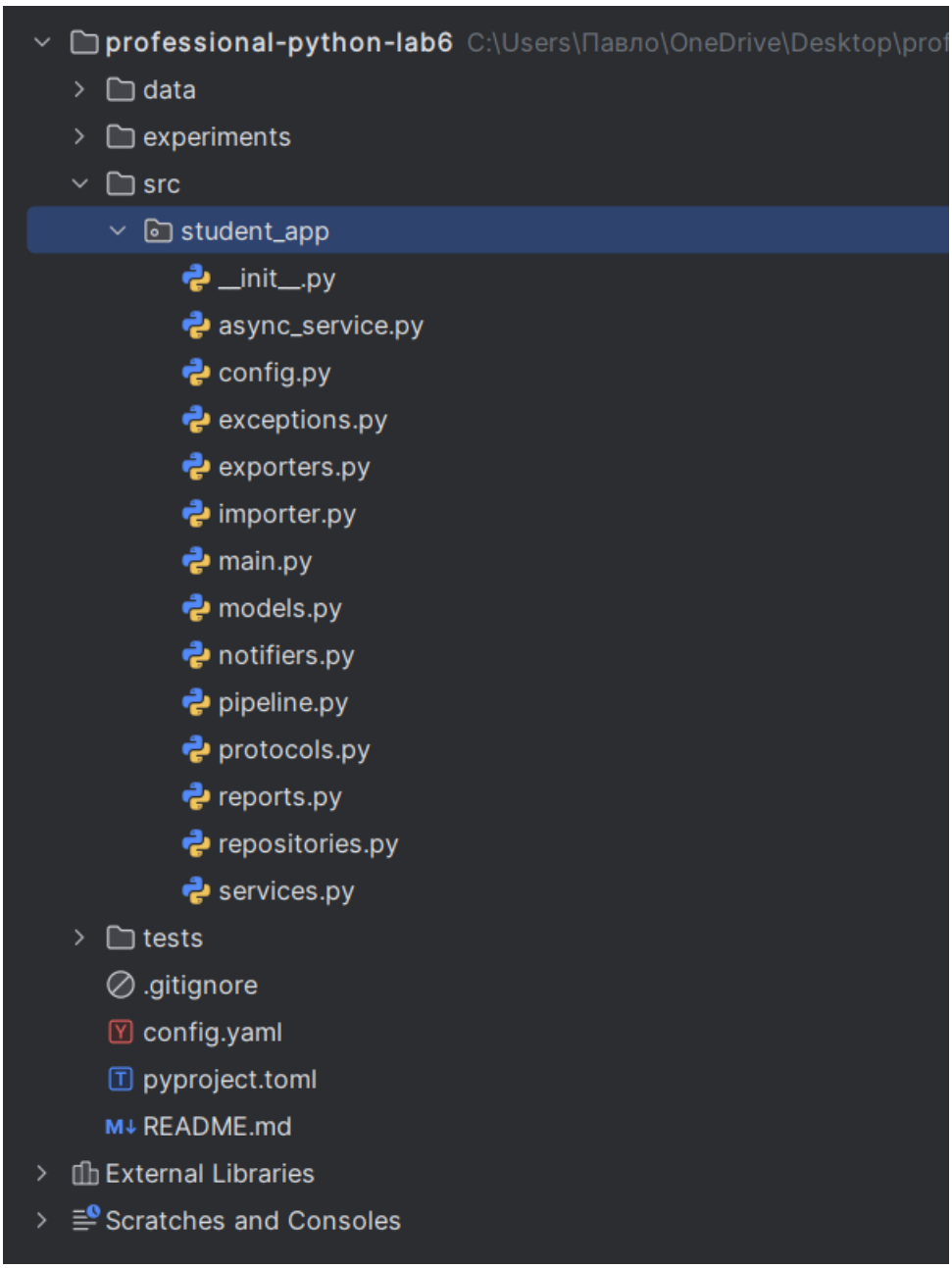


Рис. 1. Структура проєкту в PyCharm

Файл	**Test functions**	**Що перевіряє**
unit/test_models.py	8	створення, валідація, boundary, immutability, classify_grade
unit/test_selectors.py	9	пошук, фільтрація, середнє, найкращий студент, порожня колекція
unit/test_services.py	12	StudentService із Mock і fake, side_effect, autospec
unit/test_importer.py	11	CSV import, malformed record, chaining, skip_invalid
unit/test_exporters.py	5	JSON export, atomic write, failure injection
unit/test_configuration.py	10	YAML, env override, помилки конфігурації
unit/test_reports.py	4	звіт, детермінований час
unit/test_repositories.py	7	fake-репозиторії (sync/async)

Файл	Test functions	Що перевіряє
unit/test_notifiers.py	1	ConsoleNotifier (capsys)
unit/test_async_service.py	5	AsyncMock: return_value, side_effect, awaited
integration/*	10	import → validation → report → export, main(), async flow
Разом	82 функції, 113 test cases	103 unit + 10 integration (після параметризації)

5. Test matrix

Для ключових компонентів визначено мінімальну матрицю сценаріїв (normal, boundary, empty, invalid, missing data, dependency failure, repeated operation, state transition).

Категорія	Приклад у проєкті	Test
Normal	створення Student, пошук за прізвищем, середній бал	test_student_is_created_with_given_values, test_search_by_last_name_ignores_case_and_spaces, test_average_grade_of_students
Minimum boundary	average_grade = 0 (і 0.01)	test_boundary_grades_are_accepted[minimum, just-above-minimum]
Maximum boundary	average_grade = 100 (і 99.99)	test_boundary_grades_are_accepted[maximum, just-below-maximum]
Empty	порожня колекція студентів, CSV лише із заголовком, порожній експорт	test_average_grade_of_empty_collection_is_zero, test_header_only_file_gives_no_students, test_export_of_empty_collection_gives_empty_array
Invalid type/value	оцінка -0.01, 100.01, NaN, рядок, None, bool; id 0, -5, "1"	test_invalid_grades_are_rejected, test_invalid_required_fields_are_rejected
Missing data	відсутній CSV, колонка, YAML, секція конфігурації	test_missing_file_raises_import_error_with_original_cause, test_missing_columns_are_detected, test_incomplete_structure_is_rejected
Dependency failure	Notifier і репозиторій падають; async-залежність недоступна	test_failed_notification_does_not_undo_registration, test_repository_failure_is_propagated_and_nobody_is_notified, test_connection_error_of_dependency_is_propagated
Repeated operation	повторна реєстрація того самого ID, повторний експорт	test_registering_duplicate_is_rejected_without_side_effects, test_second_export_replaces_first_atomically
State transition	реєстрація змінює стан репозиторію; невдалий експорт не змінює файл	test_failed_notification_does_not_undo_registration, test_failed_export_keeps_previous_output

6. Налаштування та запуск

Залежності для розробки (pytest, pytest-cov, pytest-asyncio) описано в pyproject.toml; конфігурацію pytest та coverage теж винесено туди. Фрагмент pyproject.toml:

```
    "pytest-asyncio>=0.23",
]


[project.scripts]
student-app = "student_app.main:main"
```

```
[tool.setuptools.packages.find]
where = ["src"]

[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["src"]
addopts = ["-ra", "--strict-markers"]
asyncio_mode = "strict"
asyncio_default_fixture_loop_scope = "function"
markers = [
    "integration: integration tests (several components together)",
]

[tool.coverage.run]
branch = true
source = ["student_app"]

[tool.coverage.report]
show_missing = true
skip_covered = false
fail_under = 80
```

Встановлення та запуск усього test suite однією командою:

```
python -m pip install -e ".[dev]"
pytest -v --cov=student_app --cov-branch --cov-report=term-missing --cov-
report=html
```

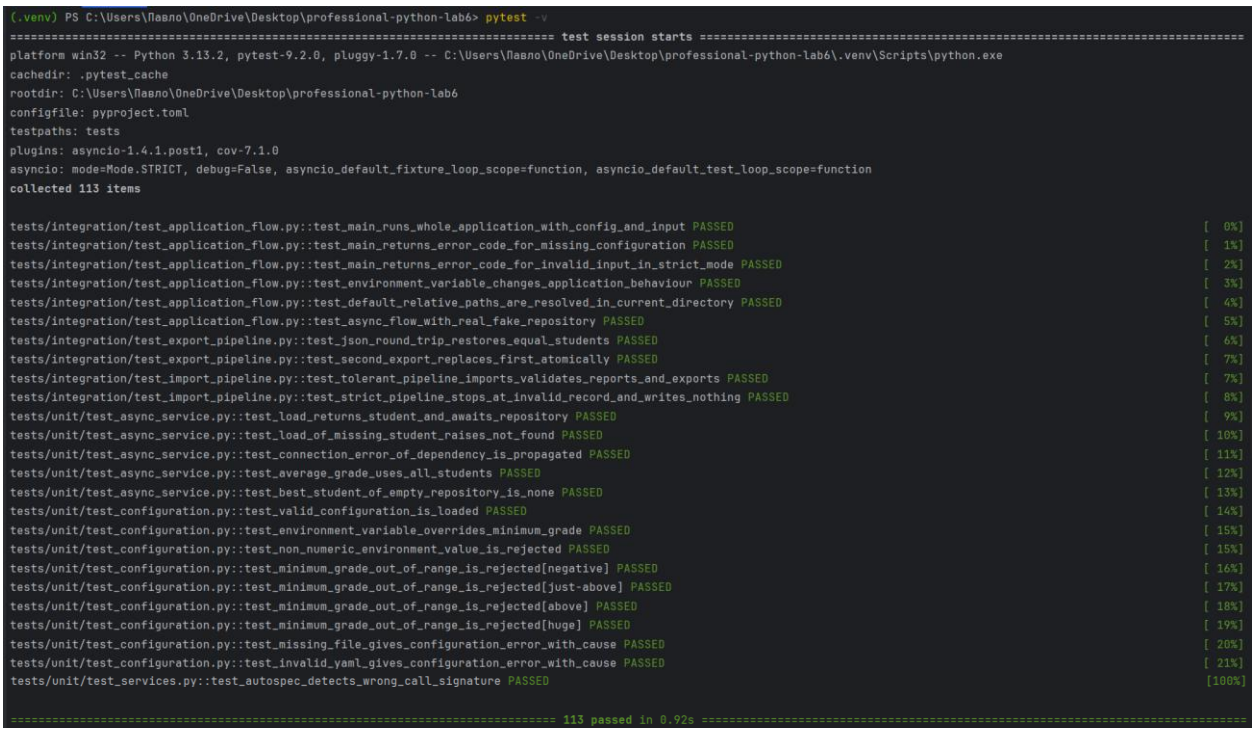


Рис. 2. Результат запуску pytest -v

Розподіл за marker: pytest -m "not integration" запускає лише unit tests, pytest -m integration — лише integration tests. Marker integration зареєстровано

в pyproject.toml, а --strict-markers гарантує, що опечатка в назві marker призведе до помилки.

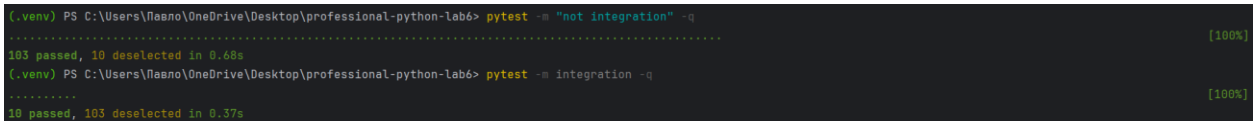


Рис. 3. Окремий запуск unit та integration tests

7. Fixtures та conftest.py

Спільні fixtures винесено в tests/conftest.py, тому в тестах вони доступні автоматично без імпорту. Реалізовано вісім fixtures: student, students, student_factory (fixture factory), repository (fake, залежить від students), notifier (Mock із spec), write_csv (повертає функцію запису CSV у tmp_path), output_dir (із yield і teardown) та autouse clean_environment, який перед кожним тестом видаляє змінні середовища застосунку (ізоляція).

```
"""Shared fixtures (pytest finds them automatically, no imports in tests)."""

from collections.abc import Callable
from pathlib import Path
from unittest.mock import Mock

import pytest

from student_app.models import Student
from student_app.protocols import Notifier
from student_app.repositories import InMemoryStudentRepository

CSV_HEADER = "student_id,first_name,last_name,group,average_grade\n"



@pytest.fixture(autouse=True)
def clean_environment(monkeypatch: pytest.MonkeyPatch) -> None:
    """Autouse: every test starts without application variables (isolation)."""
    monkeypatch.delenv("STUDENT_APP_MINIMUM_GRADE", raising=False)
    monkeypatch.delenv("APP_ENV", raising=False)



@pytest.fixture
def student() -> Student:
    return Student(1, "Anna", "Koval", "PI-21", 91.5)



@pytest.fixture
def student_factory() -> Callable[..., Student]:
    """Fixture factory: creates students with only the needed fields changed."""

    def create(
        *,
        student_id: int = 1,
        first_name: str = "Anna",
```

```
        last_name: str = "Koval",
        group: str = "PI-21",
        average_grade: float = 90.0,
    ) -> Student:
        return Student(student_id, first_name, last_name, group, average_grade)

    return create



@pytest.fixture
def students(student_factory: Callable[..., Student]) -> list[Student]:
    return [
        student_factory(student_id=1, first_name="Anna", last_name="Koval",
group="PI-21", average_grade=90),
        student_factory(student_id=2, first_name="Oleh", last_name="Melnyk",
group="PI-21", average_grade=80),
        student_factory(student_id=3, first_name="Iryna", last_name="Bondar",
group="PI-22", average_grade=95),
        student_factory(student_id=4, first_name="Andrii", last_name="Koval",
group="PI-22", average_grade=60),
    ]



@pytest.fixture
def repository(students: list[Student]) -> InMemoryStudentRepository:
    """Fake repository filled with the standard students."""
    repo = InMemoryStudentRepository()
    for item in students:
        repo.add(item)
    return repo



@pytest.fixture
def notifier() -> Mock:
    """Mock restricted to the Notifier interface."""
    return Mock(spec=Notifier)



@pytest.fixture
def write_csv(tmp_path: Path) -> Callable[..., Path]:
    """Returns a function that writes a CSV file with the standard header."""

    def write(rows: str, name: str = "students.csv") -> Path:
        path = tmp_path / name
        path.write_text(CSV_HEADER + rows, encoding="utf-8")
        return path

    return write



@pytest.fixture
def output_dir(tmp_path: Path):  # type: ignore[no-untyped-def]
    """Fixture with teardown: after the test no temporary file may remain."""
    directory = tmp_path / "output"
    directory.mkdir()

    yield directory

    leftovers = list(directory.glob("*.tmp"))
    assert not leftovers, f"temporary files were left: {leftovers}"
```

Fixture output_dir демонструє setup/teardown через yield: після кожного тесту перевіряється, що в каталозі не залишилось тимчасових .tmp файлів. Fixture student_factory дозволяє створювати об’єкти, змінюючи лише потрібні поля, а autouse fixture забезпечує, що тести не залежать від змінних середовища комп’ютера.

8. Unit tests

8.1. Parametrization, boundary testing, pytest.approx, pytest.raises

Граничні значення діапазону 0–100 перевіряються параметризованим тестом із читабельними ids; некоректні значення (нижче мінімуму, вище максимуму, NaN, рядок, None, bool) — іншим параметризованим тестом, який використовує pytest.raises із перевіркою повідомлення (match) та власного поля exception через exc_info.

```
@pytest.mark.parametrize(
    "grade",
    [
        pytest.param(0, id="minimum"),
        pytest.param(0.01, id="just-above-minimum"),
        pytest.param(50, id="middle"),
        pytest.param(99.99, id="just-below-maximum"),
        pytest.param(100, id="maximum"),
    ],
)
def test_boundary_grades_are_accepted(student_factory, grade):
    assert student_factory(average_grade=grade).average_grade ==
pytest.approx(grade)



@pytest.mark.parametrize(
    "grade",
    [-0.01, 100.01, float("nan"), "90", None, True],
    ids=["below-minimum", "above-maximum", "nan", "string", "none", "bool"],
)
def test_invalid_grades_are_rejected(student_factory, grade):
    with pytest.raises(ValidationError, match="between 0 and 100") as exc_info:
        student_factory(average_grade=grade)

    assert exc_info.value.field == "average_grade"



@pytest.mark.parametrize(
    ("kwargs", "field"),
    [
        ({"student_id": 0}, "student_id"),
        ({"student_id": -5}, "student_id"),
        ({"student_id": "1"}, "student_id"),
        ({"first_name": ""}, "first_name"),
        ({"last_name": "   "}, "last_name"),
        ({"group": ""}, "group"),
    ],
```

```
    ids=["zero-id", "negative-id", "string-id", "empty-first-name", "blank-last-
name", "empty-group"],
)
def test_invalid_required_fields_are_rejected(student_factory, kwargs, field):
    with pytest.raises(ValidationError) as exc_info:
        student_factory(**kwargs)

    assert exc_info.value.field == field



@pytest.mark.parametrize(
    ("grade", "expected"),
    [
        (100, "excellent"),
        (90, "excellent"),
        (89.99, "passed"),
        (60, "passed"),
        (59.99, "failed"),
        (0, "failed"),
    ],
)
def test_classify_grade_covers_every_branch(grade, expected):
    assert classify_grade(grade) == expected
```

Для порівняння дійсних чисел використано pytest.approx, оскільки перевірка через == небезпечна через особливості представлення чисел із плаваючою комою (наприклад, середнє значення 81.25 для чотирьох оцінок).

```
def test_average_grade_of_students(students):
    assert calculate_average(students) == pytest.approx(81.25)



def test_average_grade_of_empty_collection_is_zero():
    assert calculate_average([]) == 0.0



def test_best_student_with_equal_grades_is_the_first_one(student_factory):
    first = student_factory(student_id=1, average_grade=70)
    second = student_factory(student_id=2, average_grade=70)

    assert find_best_student([first, second]) is first
```

8.2. Mock, return_value, side_effect, interaction assertions

StudentService залежить від двох abstractions, які підміняються у тестах. Mock(spec=...) обмежує інтерфейс mock-а справжнім Protocol, return_value задає результат виклику, side_effect моделює збій залежності, а assert_called_once_with / assert_not_called перевіряють взаємодію. Особливо важливий негативний сценарій: якщо репозиторій упав, сповіщення не повинно бути надіслано; якщо впало сповіщення, студент усе одно зареєстрований (перевіряється станом fake-репозиторію).

```
"""StudentService with test doubles: Mock for interaction, fake for state."""


from unittest.mock import Mock, create_autospec


import pytest


from student_app.exceptions import NotificationError, ValidationError
from student_app.protocols import Notifier, StudentRepository
from student_app.repositories import InMemoryStudentRepository
from student_app.services import StudentService



@pytest.fixture
def mock_repository() -> Mock:
    repository = Mock(spec=StudentRepository)
    repository.get.return_value = None
```

```
def test_register_stores_student_and_sends_notification(student, mock_repository,
notifier):
    service = StudentService(mock_repository, notifier)


    delivered = service.register(student)


    assert delivered is True
    mock_repository.get.assert_called_once_with(1)
    mock_repository.add.assert_called_once_with(student)
    notifier.send.assert_called_once_with("Anna Koval", "Registered in group PI-
21")



def test_registering_duplicate_is_rejected_without_side_effects(student,
mock_repository, notifier):
    mock_repository.get.return_value = student
    service = StudentService(mock_repository, notifier)


    with pytest.raises(ValidationError, match="already exists"):
        service.register(student)


    mock_repository.add.assert_not_called()
    notifier.send.assert_not_called()



def test_failed_notification_does_not_undo_registration(student, notifier):
    repository = InMemoryStudentRepository()
    notifier.send.side_effect = NotificationError("SMTP is down")
    service = StudentService(repository, notifier)


    delivered = service.register(student)


    assert delivered is False
    assert repository.get(1) == student
    notifier.send.assert_called_once()



def test_repository_failure_is_propagated_and_nobody_is_notified(student,
mock_repository, notifier):
    mock_repository.add.side_effect = RuntimeError("database is down")
```

```
    service = StudentService(mock_repository, notifier)

    with pytest.raises(RuntimeError, match="database is down"):
        service.register(student)

    notifier.send.assert_not_called()
```

autospec суворіший за звичайний Mock(spec=...): він перевіряє також сигнатуру викликів. Два тести порівнюють їх поведінку.

```
def test_plain_mock_with_spec_does_not_check_call_signature():
    notifier = Mock(spec=Notifier)

    notifier.send("only recipient")  # wrong call, but the mock accepts it

    notifier.send.assert_called_once_with("only recipient")



def test_autospec_detects_wrong_call_signature():
    notifier = create_autospec(Notifier, instance=True)

    with pytest.raises(TypeError):
        notifier.send("only recipient")
```

8.3. tmp_path, перевірка CSV import та exception chaining

```
def test_valid_csv_is_imported(write_csv):
    path = write_csv("1,Anna,Koval,PI-21,90\n2,Oleh,Melnyk,PI-21,80.5\n")

    result = list(read_students(path))

    assert [s.student_id for s in result] == [1, 2]
    assert result[1].average_grade == pytest.approx(80.5)



def test_non_numeric_grade_raises_error_with_line_number_and_cause(write_csv):
    path = write_csv("1,Anna,Koval,PI-21,abc\n")

    with pytest.raises(ImportDataError, match="average_grade") as exc_info:
        list(read_students(path))

    assert exc_info.value.line_number == 2
    assert isinstance(exc_info.value.__cause__, ValueError)



def test_grade_out_of_range_keeps_validation_error_as_cause(write_csv):
    path = write_csv("1,Anna,Koval,PI-21,105\n")

    with pytest.raises(ImportDataError) as exc_info:
        list(read_students(path))

    assert exc_info.value.line_number == 2
    assert isinstance(exc_info.value.__cause__, ValidationError)
```

```
def test_skip_invalid_collects_errors_and_keeps_valid_students(write_csv):
    path = write_csv("1,Anna,Koval,PI-21,90\nx,Bad,Id,PI-21,80\n3,Iryna,Bondar,PI-
22,95\n4,Oleh,Melnyk,PI-21,\n")
    errors: list[ImportDataError] = []

    result = list(read_students(path, skip_invalid=True, errors=errors))

    assert [s.student_id for s in result] == [1, 3]
    assert [e.line_number for e in errors] == [3, 5]



def test_strict_mode_yields_valid_records_before_the_first_error(write_csv):
    path = write_csv("1,Anna,Koval,PI-21,90\nx,Bad,Id,PI-21,80\n")
    stream = read_students(path)

    assert next(stream).student_id == 1
    with pytest.raises(ImportDataError):
        next(stream)
```

Усі файлові тести створюють файли в ізольованому каталозі tmp_path (через fixture write_csv), тому не залишають сміття в репозиторії та не залежать від попередніх запусків. Тест test_non_numeric_grade... також перевіряє ланцюжок винятків: __cause__ є ValueError.

8.4. JSON export, atomic output та failure injection

```
def test_export_writes_utf8_json_array(output_dir, student_factory):
    path = output_dir / "students.json"
    items = [
        student_factory(student_id=1, first_name="Олена", last_name="Коваль"),
        student_factory(student_id=2),
    ]

    count = export_students_json(items, path)

    data = json.loads(path.read_text(encoding="utf-8"))
    assert count == 2
    assert [row["student_id"] for row in data] == [1, 2]
    assert data[0]["first_name"] == "Олена"



def test_failed_export_keeps_previous_output(output_dir, student_factory):
    path = output_dir / "students.json"
    path.write_text("old valid data", encoding="utf-8")

    def broken_stream():
        yield student_factory(student_id=1)
        raise RuntimeError("stream failed")

    with pytest.raises(RuntimeError, match="stream failed"):
        export_students_json(broken_stream(), path)

    assert path.read_text(encoding="utf-8") == "old valid data"
```

```
    assert not path.with_name("students.json.tmp").exists()



def test_os_error_becomes_export_error_and_keeps_previous_output(output_dir,
student, monkeypatch):
    path = output_dir / "students.json"
    path.write_text("old valid data", encoding="utf-8")

    def disk_full(self, *args, **kwargs):
        raise OSError("disk is full")

    monkeypatch.setattr(Path, "write_text", disk_full)

    with pytest.raises(ExportError, match="Cannot write") as exc_info:
        export_students_json([student], path)

    monkeypatch.undo()
    assert isinstance(exc_info.value.__cause__, OSError)
    assert path.read_text(encoding="utf-8") == "old valid data"
```

Другий тест вводить збій у потік даних (generator піднімає RuntimeError), третій за допомогою monkeypatch підміняє Path.write_text так, щоб запис завжди завершувався OSError («disk is full»). В обох випадках перевіряється, що попередній коректний результат не пошкоджено, а тимчасовий файл видалено.

8.5. Configuration, monkeypatch, детермінований час

monkeypatch.setenv/delenv керує змінними середовища, а monkeypatch.setattr підміняє функцію utcnow фіксованим моментом, завдяки чому звіт із часом генерації стає детермінованим. Після тесту всі зміни автоматично скасовуються.

```
def test_environment_variable_overrides_minimum_grade(config_file, monkeypatch):
    monkeypatch.setenv(ENV_MINIMUM_GRADE, "75")

    assert load_config(config_file()).minimum_grade == pytest.approx(75)



@pytest.mark.parametrize("grade", [-1, 100.5, 101, 1000], ids=["negative", "just-
above", "above", "huge"])
def test_minimum_grade_out_of_range_is_rejected(config_file, grade):
    text = VALID.replace("minimum_grade: 60", f"minimum_grade: {grade}")

    with pytest.raises(ConfigurationError, match="between 0 and 100"):
        load_config(config_file(text))



def test_invalid_yaml_gives_configuration_error_with_cause(config_file):
    with pytest.raises(ConfigurationError, match="Invalid YAML") as exc_info:
        load_config(config_file("a: [unclosed"))
```

```
    assert isinstance(exc_info.value.__cause__, yaml.YAMLError)


def test_default_environment_name():
    assert get_environment() == "development"


def test_environment_name_from_variable(monkeypatch):
    monkeypatch.setenv("APP_ENV", "testing")

    assert get_environment() == "testing"
```

```
from datetime import datetime, timezone

import pytest

from student_app import reports


@pytest.fixture
def frozen_time(monkeypatch):
    """Deterministic time: the clock function is replaced by monkeypatch."""
    moment = datetime(2026, 11, 8, 12, 0, tzinfo=timezone.utc)
    monkeypatch.setattr(reports, "utcnow", lambda: moment)
    return moment


def test_report_describes_all_students(students, frozen_time):
    report = reports.build_report(students)

    assert report["generated_at"] == "2026-11-08T12:00:00+00:00"
    assert report["count"] == 4
    assert report["average"] == pytest.approx(81.25)
    assert report["best"]["student_id"] == 3
    assert report["by_group"] == {"PI-21": 2, "PI-22": 2}


def test_report_filters_by_minimum_grade(students, frozen_time):
    report = reports.build_report(students, minimum_grade=80)

    assert report["count"] == 3
    assert report["by_group"] == {"PI-21": 2, "PI-22": 1}


def test_report_of_empty_collection(frozen_time):
    report = reports.build_report([])

    assert report["count"] == 0
```

8.6. AsyncMock

У проєкті є асинхронна залежність (AsyncStudentRepository), тому використано AsyncMock. Тести позначено pytest.mark.asyncio (pytest-asyncio у strict mode). Для awaited-викликів застосовано assert_awaited_once_with та assert_awaited_once.

```
from unittest.mock import AsyncMock

import pytest

from student_app.async_service import AsyncStudentService
from student_app.exceptions import NotFoundError
from student_app.protocols import AsyncStudentRepository



@pytest.fixture
def async_repository() -> AsyncMock:
    return AsyncMock(spec=AsyncStudentRepository)



@pytest.mark.asyncio
async def test_load_returns_student_and_awaits_repository(async_repository,
student):
    async_repository.get.return_value = student

    result = await AsyncStudentService(async_repository).load(1)

    assert result is student
    async_repository.get.assert_awaited_once_with(1)



@pytest.mark.asyncio
async def test_load_of_missing_student_raises_not_found(async_repository):
    async_repository.get.return_value = None

    with pytest.raises(NotFoundError, match="not found"):
        await AsyncStudentService(async_repository).load(99)



@pytest.mark.asyncio
async def test_connection_error_of_dependency_is_propagated(async_repository):
    async_repository.get.side_effect = ConnectionError("Network error")

    with pytest.raises(ConnectionError, match="Network error"):
        await AsyncStudentService(async_repository).load(1)



@pytest.mark.asyncio
async def test_average_grade_uses_all_students(async_repository, students):
    async_repository.all.return_value = students

    result = await AsyncStudentService(async_repository).average_grade()

    assert result == pytest.approx(81.25)
```

```
    async_repository.all.assert_awaited_once()
```

9. Integration tests

Integration tests (marker integration) використовують справжні тимчасові файли, YAML, CSV та JSON-парсер і перевіряють взаємодію компонентів без підміни внутрішньої логіки. Реалізовано 10 тестів у трьох файлах. Основний — повний pipeline CSV → валідація → фільтрація → звіт → JSON:

```
import json

import pytest

from student_app.config import AppConfig
from student_app.exceptions import ImportDataError
from student_app.pipeline import process

pytestmark = pytest.mark.integration

ROWS = (
    "1,Anna,Koval,PI-21,90\n"
    "2,Oleh,Melnyk,PI-21,55\n"
    "3,Iryna,Bondar,PI-22,x\n"
    "4,,Boyko,PI-22,70\n"
    "5,Petro,Novak,PI-23,70\n"
)



def test_tolerant_pipeline_imports_validates_reports_and_exports(write_csv,
output_dir):
    config = AppConfig(60.0, True, output_dir / "students.json")

    result = process(write_csv(ROWS), config)

    exported = json.loads((output_dir / "students.json").read_text(encoding="utf-
8"))
    assert [row["student_id"] for row in exported] == [1, 5]
    assert result.exported == 2
    assert [e.line_number for e in result.errors] == [4, 5]
    assert result.report["count"] == 2
    assert result.report["average"] == pytest.approx(80.0)
    assert result.report["best"]["student_id"] == 1
    assert result.report["by_group"] == {"PI-21": 1, "PI-23": 1}



def test_strict_pipeline_stops_at_invalid_record_and_writes_nothing(write_csv,
output_dir):
    config = AppConfig(60.0, False, output_dir / "students.json")

    with pytest.raises(ImportDataError, match="average_grade"):
        process(write_csv(ROWS), config)

    assert not (output_dir / "students.json").exists()
```

Інші integration tests: JSON round trip (експорт і повторне відновлення об’єктів), повторний експорт, який атомарно замінює попередній файл, запуск усього застосунку через main() із YAML та CSV, коди завершення при помилках, вплив змінної середовища, відносні шляхи через monkeypatch.chdir та асинхронний flow із реальним fake-репозиторієм.

```
def test_main_runs_whole_application_with_config_and_input(tmp_path, write_csv):
    config = write_config(tmp_path)

    code = main(["--config", str(config), "--input", str(write_csv(ROWS))])

    data = json.loads((tmp_path / "result.json").read_text(encoding="utf-8"))
    assert code == 0
    assert [row["student_id"] for row in data] == [1, 2]



def test_environment_variable_changes_application_behaviour(tmp_path, write_csv,
monkeypatch):
    monkeypatch.setenv("STUDENT_APP_MINIMUM_GRADE", "95")
    config = write_config(tmp_path)

    code = main(["--config", str(config), "--input", str(write_csv(ROWS))])

    data = json.loads((tmp_path / "result.json").read_text(encoding="utf-8"))
    assert code == 0
    assert [row["student_id"] for row in data] == [2]



@pytest.mark.asyncio
async def test_async_flow_with_real_fake_repository(students):
    service = AsyncStudentService(AsyncInMemoryStudentRepository(students))

    assert (await service.load(2)).last_name == "Melnyk"
    assert await service.average_grade() == pytest.approx(81.25)
    best = await service.best_student()
    assert best is not None and best.student_id == 3
    with pytest.raises(NotFoundError):
        await service.load(99)
```

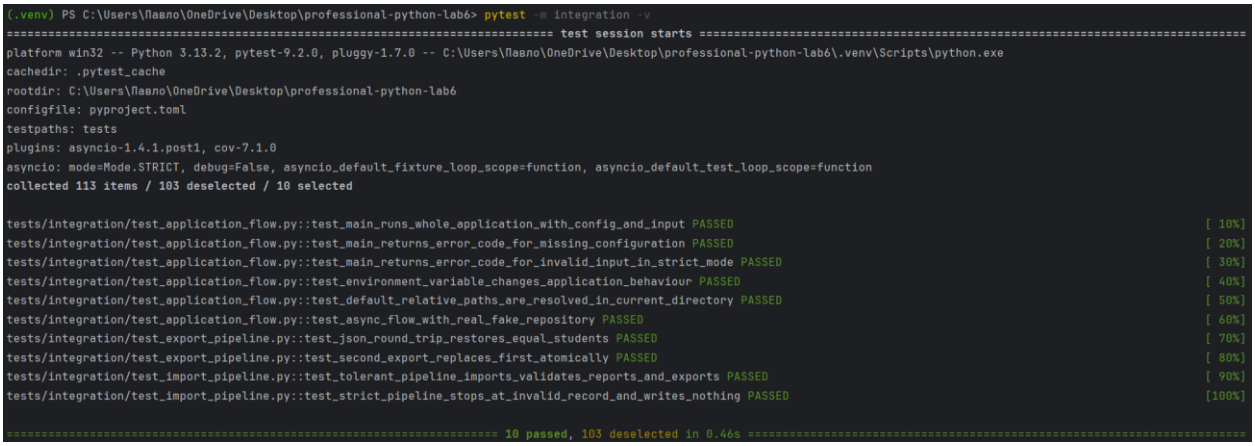


Рис. 4. Запуск integration tests

10. Coverage

Coverage вимірювався командою pytest --cov=student_app --cov-branch --cov-report=term-missing --cov-report=html. У pyproject.toml задано мінімальний поріг fail_under = 80, тому запуск завершиться помилкою, якщо покриття буде нижчим. Рядок if __name__ == "__main__" у main.py виключено директивою # pragma: no cover, оскільки запуск через python -m перевіряється поза pytest.

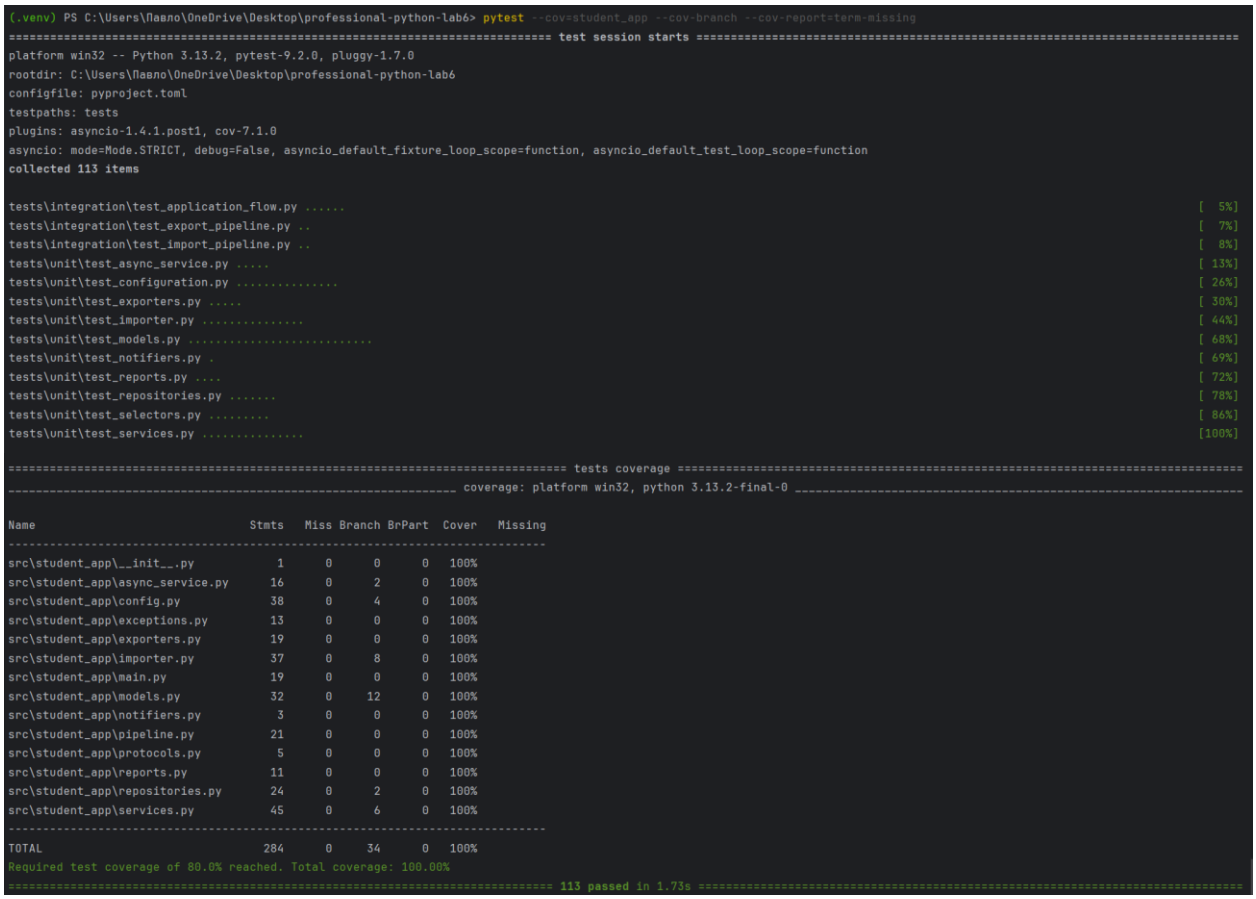


Рис. 5. Coverage report у терміналі (statement та branch, колонка Missing)

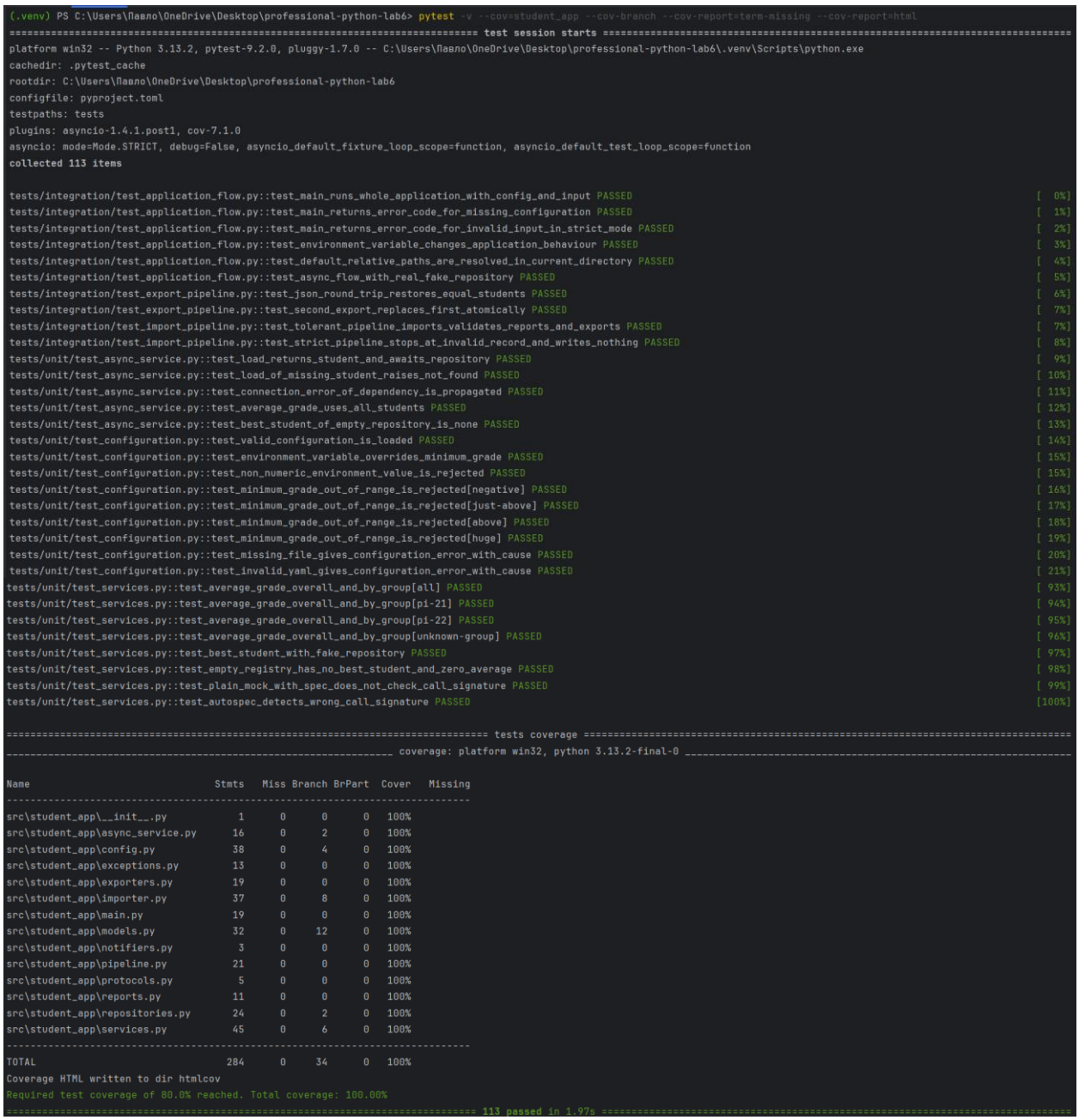


Рис. 6. Фрагмент HTML coverage report

10.1. Аналіз недостатньо протестованих paths

Під час аналізу покриття початковий набір тестів виявив дві групи paths, яких тести не торкались:

- ConsoleNotifier (notifiers.py) — жоден тест не імпортував модуль, тому його покриття дорівнювало 0%. Додано tests/unit/test_notifiers.py, який через fixture capsys перевіряє, що саме виводиться.
- InMemoryStudentRepository (repositories.py): гілка відхилення дубліката ID, метод remove (обидва результати) та __len__ не мали

жодного тесту. Додано tests/unit/test_repositories.py, у тому числі перевірку, що all() повертає копію, і тести асинхронного fake.

Для демонстрації на захисті можна тимчасово перейменувати tests/unit/test_repositories.py і повторно запустити coverage: у колонці Missing з'являться саме ці рядки. Також у branch coverage показано, що кожен if має перевірені обидві гілки (наприклад, skip_invalid True/False, errors is None / не None, group is None / не None у average_grade).

11. Тести та знайдені дефекти

Тести повинні виявляти дефекти. Для демонстрації у функції calculate_average (services.py) навмисно внесено помилку: ділення на len(grades) + 1 замість len(grades). Після цього падають вісім тестів, які перевіряють результат на різних рівнях (unit, сервіс, звіт, async, integration):

```
FAILED tests/unit/test_selectors.py::test_average_grade_of_students
FAILED tests/unit/test_services.py::test_average_grade_overall_and_by_group[all]
FAILED tests/unit/test_services.py::test_average_grade_overall_and_by_group[pi-21]
FAILED tests/unit/test_services.py::test_average_grade_overall_and_by_group[pi-22]
FAILED tests/unit/test_reports.py::test_report_describes_all_students
FAILED tests/unit/test_async_service.py::test_average_grade_uses_all_students
FAILED
tests/integration/test_import_pipeline.py::test_tolerant_pipeline_imports_validate
s_reports_and_exports
FAILED
tests/integration/test_application_flow.py::test_async_flow_with_real_fake_reposit
ory
```

Тест test_average_grade_of_empty_collection_is_zero при цьому лишається зеленим: порожня колекція обробляється окремою гілкою, тому змінений рядок не виконується. Після виправлення дефекту всі тести знову проходять.



Рис. 7. Failing tests після навмисної поломки calculate_average

12. Експерименти

12.1. Експеримент 1. Weak проти strong tests

Слабкий тест виконує код, але нічого не перевіряє; сильний перевіряє результат. Обидва тести викликають функцію з навмисним дефектом (ділення на len + 1).

```
"""Function with a deliberate defect for experiments 1 and 2."""


def buggy_average(values: list[float]) -> float:
    """Wrong on purpose: divides by len(values) + 1."""
    return sum(values) / (len(values) + 1)
```

```
"""Experiments 1 and 2: a weak test executes code, a strong test verifies the
result.

Run:  pytest experiments/test_exp1_2_weak_vs_strong.py -k weak
      pytest experiments/test_exp1_2_weak_vs_strong.py -k strong        (fails on
purpose)
"""

import pytest

from buggy import buggy_average


def test_average_weak():
    buggy_average([10, 20, 30])  # the code is executed, but nothing is checked
```

```
def test_average_strong():
    assert buggy_average([10, 20, 30]) == pytest.approx(20)  # real result is 15.0
```

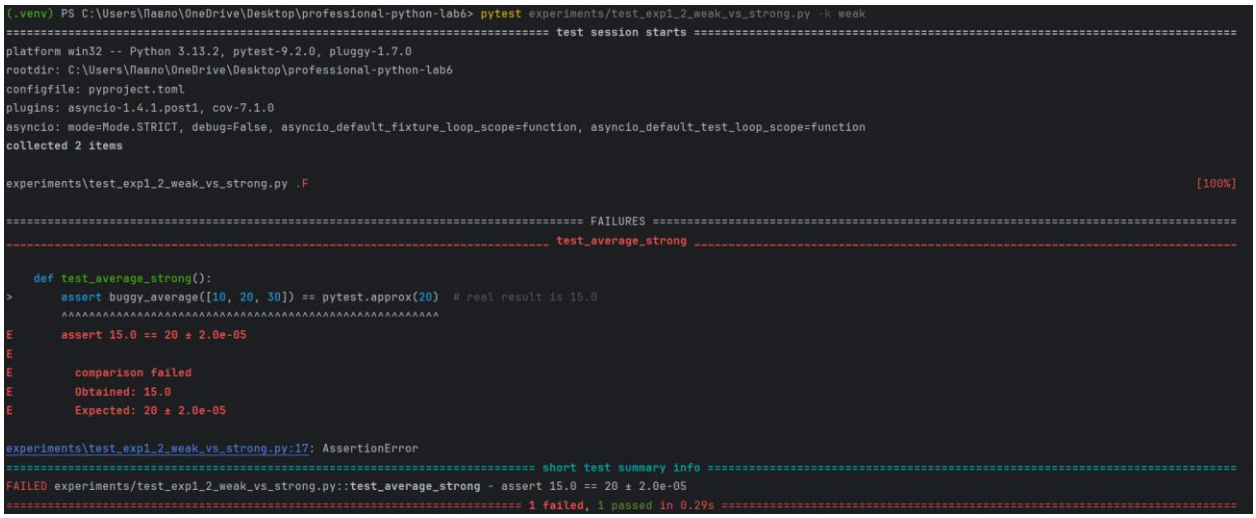


Рис. 8. Weak test проходить попри дефект

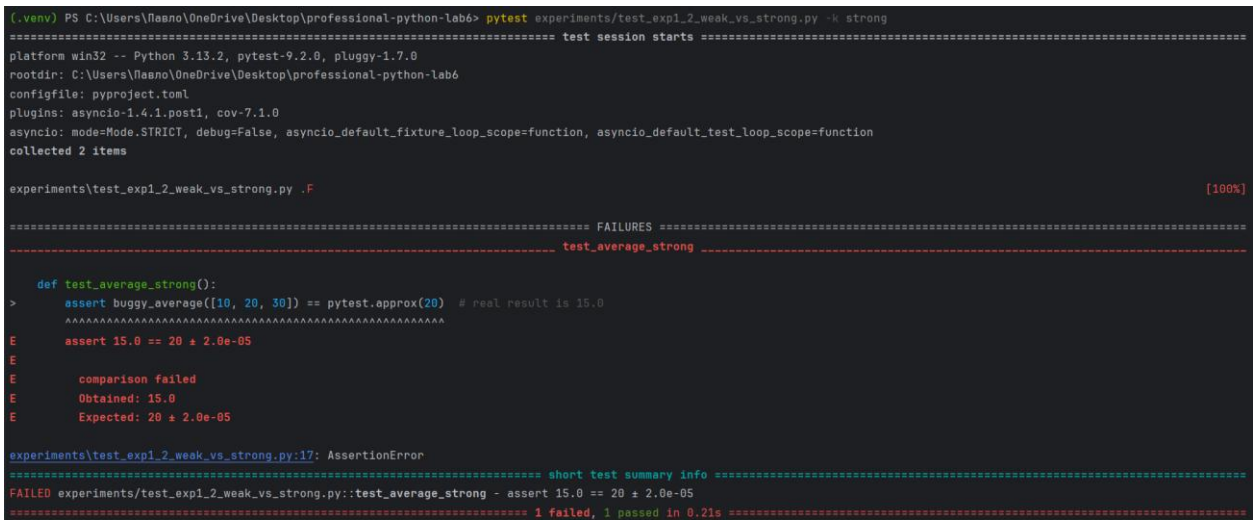


Рис. 9. Strong test виявляє дефект (очікувано FAILED: 15.0 замість 20)

Висновок: execution ≠ verification. Слабкий тест успішно виконується, хоча функція повертає 15.0 замість 20; помилку знаходить лише тест із осмисленим assert.

12.2. Експеримент 2. Coverage без assertions

Слабкий тест виконує всі рядки функції buggy_average, тому її statement coverage становить 100%, хоча результат неправильний.

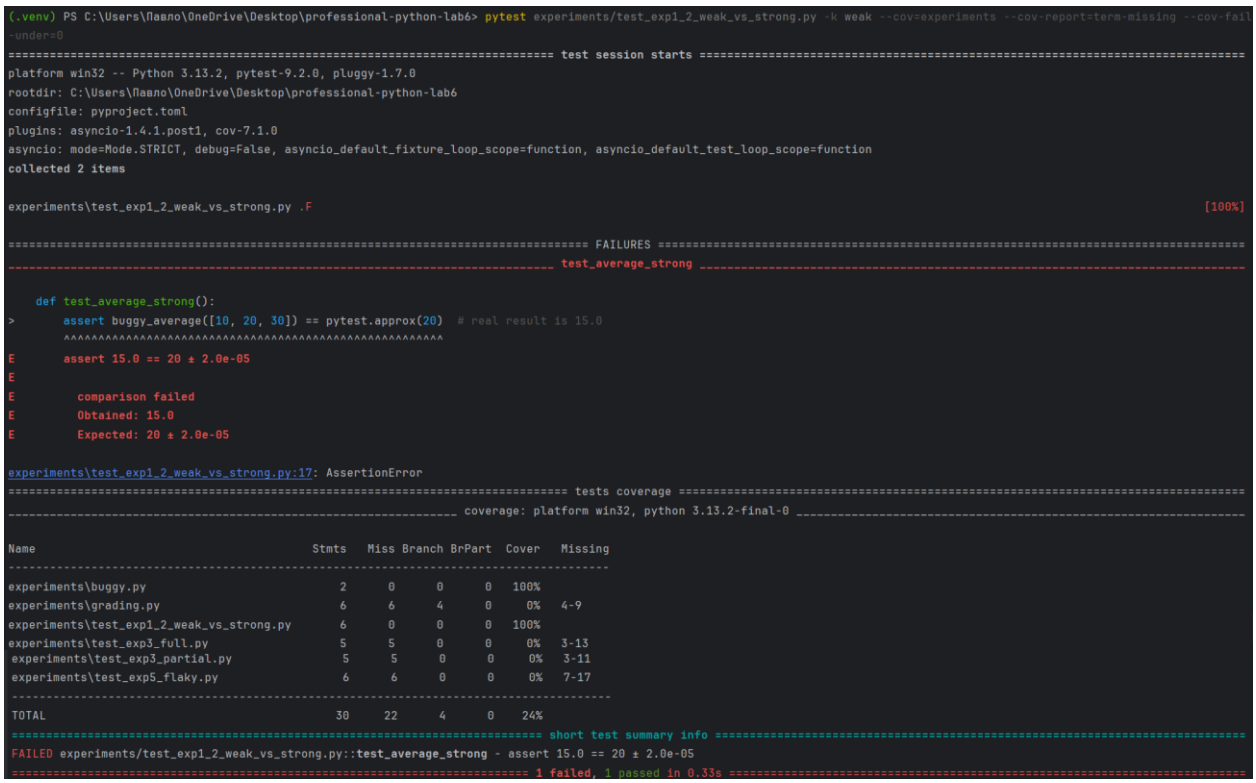


Рис. 10. 100% coverage при слабкому тесті

Висновок: coverage показує лише, що код виконувався, але не що результат перевірено. Якість assertions важливіша за відсоток покриття.

12.3. Експеримент 3. Branch coverage

```
"""Function with three branches for experiment 3."""


def classify_grade(grade: float) -> str:
    if grade >= 90:
        return "excellent"
    if grade >= 60:
        return "passed"
    return "failed"
```

Спочатку перевіряються лише значення 95 і 70 (файл test_exp3_partial.py): гілка return "failed" не виконується, і branch coverage показує пропущений перехід. Потім додається значення 50 (test_exp3_full.py) — усі три гілки покрито.

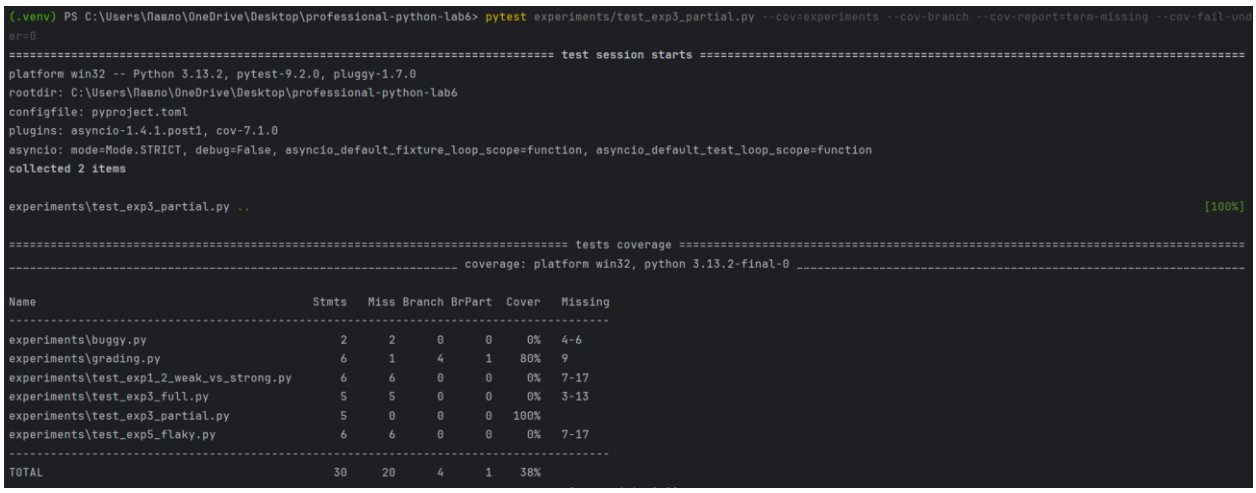


Рис. 11. Branch coverage для двох тестів (пропущена гілка)

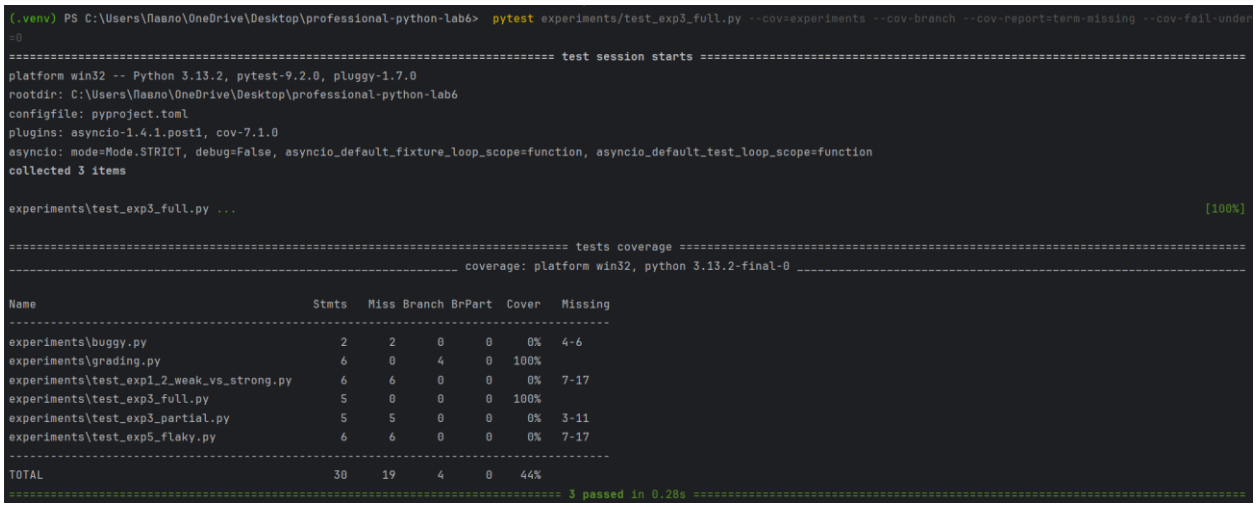


Рис. 12. Branch coverage після додавання значення 50

12.4. Експеримент 4. Mock проти fake

Для StudentRepository використано обидва підходи: Mock(spec=StudentRepository) у тестах register / find_by_id / find_by_last_name і fake InMemoryStudentRepository у тестах пошуку, середнього, найкращого студента та збою сповіщення.

Критерій	Mock	Fake (InMemoryStudentRepository)
Читабельність	потрібно налаштовувати return_value/side_effect у кожному тесті	тест читається як сценарій: додати студента, отримати результат
State testing	незручно: стан не зберігається	природно: перевіряється repository.get(1) після операції
Interaction testing	зручно: assert_called_once_with, assert_not_called	неможливо без додаткового коду
Повторне використання	налаштовується окремо в кожному тесті	один клас і fixture repository для багатьох тестів
Зв’язаність тесту з реалізацією	висока: тест знає, які методи і скільки разів викликаються	низька: перевіряється лише результат

Висновок: Mock доцільний для перевірки взаємодії та збоїв (side_effect), fake — для перевірки стану й складних сценаріїв. Тому в suite поєднано обидва підходи.

12.5. Експеримент 5. Flaky test

```
"""Experiment 5: flaky and deterministic versions of the same check.

Run the bad test many times (PowerShell):
    1..10 | ForEach-Object { pytest experiments/test_exp5_flaky.py -k bad -q }
"""

import random



def test_random_bad():
    assert random.randint(1, 2) == 1  # passes only in about half of the runs



def test_random_deterministic(monkeypatch):
    monkeypatch.setattr(random, "randint", lambda a, b: 1)

    assert random.randint(1, 2) == 1  # always passes: the randomness is
controlled
```

```
(.venv) PS C:\Users\Павло\OneDrive\Desktop\professional-python-lab6> 1..10 | ForEach-Object { pytest experiments/test_exp5_flaky.py -k bad -q }
.                                                                        [100%]
1 passed, 1 deselected in 0.03s
F                                                                        [100%]
================================== FAILURES ===================================
_____________________________ test_random_bad _____________________________

    def test_random_bad():
>       assert random.randint(1, 2) == 1  # passes only in about half of the runs
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
E       assert 2 == 1
E        +  where 2 = randint(1, 2)
E        +    where randint = random.randint

experiments\test_exp5_flaky.py:11: AssertionError
=========================== short test summary info ===========================
FAILED experiments/test_exp5_flaky.py::test_random_bad - assert 2 == 1
1 failed, 1 deselected in 0.25s
F                                                                        [100%]
================================== FAILURES ===================================
_____________________________ test_random_bad _____________________________

    def test_random_bad():
>       assert random.randint(1, 2) == 1  # passes only in about half of the runs
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
E       assert 2 == 1
E        +  where 2 = randint(1, 2)
E        +    where randint = random.randint

experiments\test_exp5_flaky.py:11: AssertionError
=========================== short test summary info ===========================
FAILED experiments/test_exp5_flaky.py::test_random_bad - assert 2 == 1
1 failed, 1 deselected in 0.22s
.                                                                        [100%]
1 passed, 1 deselected in 0.02s
F                                                                        [100%]
================================== FAILURES ===================================
_____________________________ test_random_bad _____________________________

    def test_random_bad():
>       assert random.randint(1, 2) == 1  # passes only in about half of the runs
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
E       assert 2 == 1
E        +  where 2 = randint(1, 2)
E        +    where randint = random.randint

experiments\test_exp5_flaky.py:11: AssertionError
=========================== short test summary info ===========================
FAILED experiments/test_exp5_flaky.py::test_random_bad - assert 2 == 1
1 failed, 1 deselected in 0.20s
F                                                                        [100%]
================================== FAILURES ===================================
_____________________________ test_random_bad _____________________________

    def test_random_bad():
>       assert random.randint(1, 2) == 1  # passes only in about half of the runs
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
E       assert 2 == 1
E        +  where 2 = randint(1, 2)
E        +    where randint = random.randint

experiments\test_exp5_flaky.py:11: AssertionError
=========================== short test summary info ===========================
FAILED experiments/test_exp5_flaky.py::test_random_bad - assert 2 == 1
1 failed, 1 deselected in 0.21s
F                                                                        [100%]
================================== FAILURES ===================================
================================== FAILURES ===================================
_____________________________ test_random_bad _____________________________

    def test_random_bad():
>       assert random.randint(1, 2) == 1  # passes only in about half of the runs
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
E       assert 2 == 1
E        +  where 2 = randint(1, 2)
E        +    where randint = random.randint

experiments\test_exp5_flaky.py:11: AssertionError
=========================== short test summary info ===========================
FAILED experiments/test_exp5_flaky.py::test_random_bad - assert 2 == 1
1 failed, 1 deselected in 0.21s
F                                                                        [100%]
================================== FAILURES ===================================
_____________________________ test_random_bad _____________________________

    def test_random_bad():
>       assert random.randint(1, 2) == 1  # passes only in about half of the runs
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
E       assert 2 == 1
E        +  where 2 = randint(1, 2)
E        +    where randint = random.randint

experiments\test_exp5_flaky.py:11: AssertionError
=========================== short test summary info ===========================
FAILED experiments/test_exp5_flaky.py::test_random_bad - assert 2 == 1
1 failed, 1 deselected in 0.20s
.                                                                        [100%]
1 passed, 1 deselected in 0.03s
.                                                                        [100%]
1 passed, 1 deselected in 0.02s
```

Рис. 13. Багаторазовий запуск flaky test (PASSED і FAILED у різних запусках)

Тест test_random_bad проходить приблизно в половині запусків, тому він непередбачуваний: падіння не означає, що змінився код, а успіх не доводить, що він правильний. Flaky tests знецінюють suite, бо розробники перестають довіряти результатам. Детермінована версія підміняє генератор випадкових чисел через monkeypatch і проходить завжди.

12.6. Експеримент 6. Ізоляція tmp_path

Файлові тести запущено кілька разів поспіль, після чого перевірено стан репозиторію.

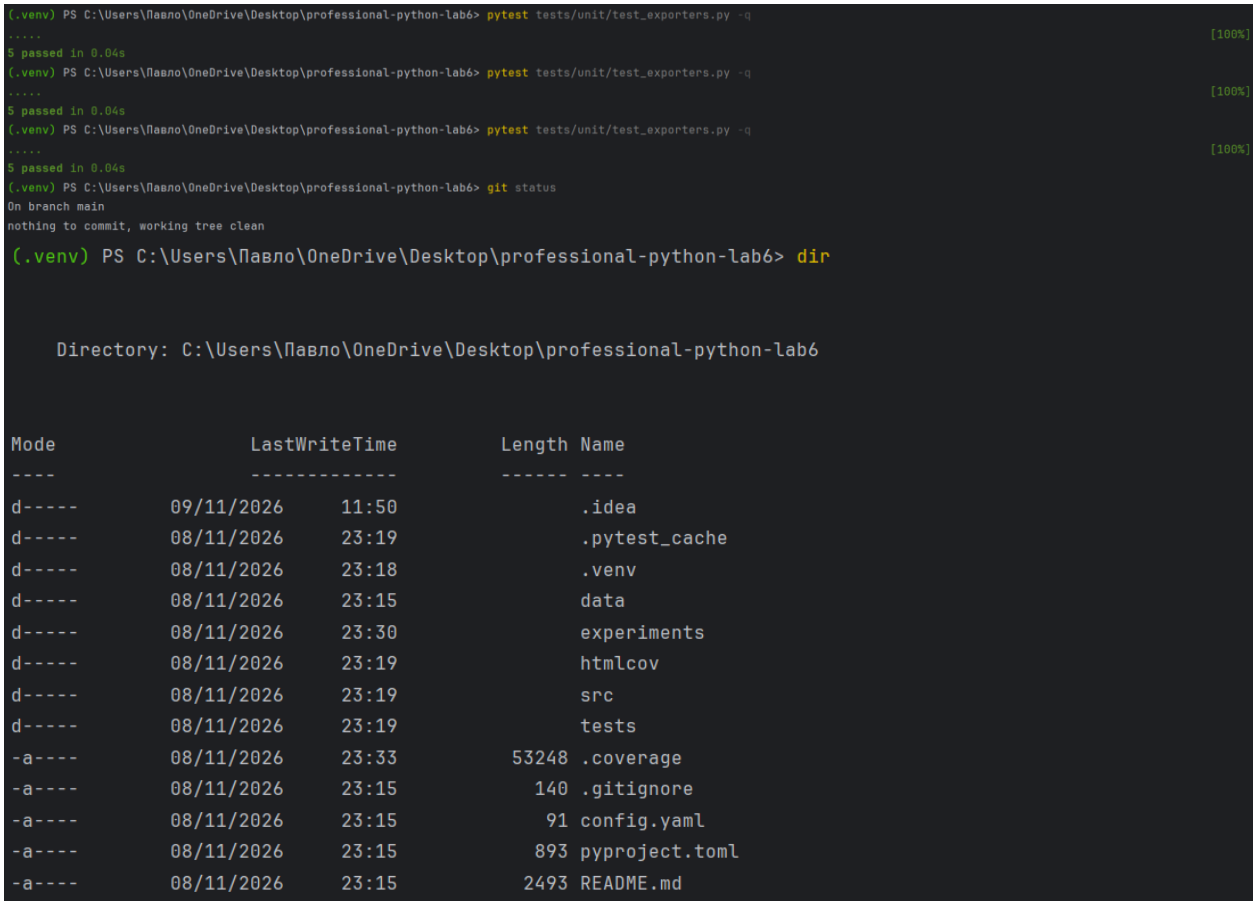


Рис. 14. Повторні запуски файлових tests та git status

Після запусків git status показує чисте дерево (не рахуючи кешів, які ігнорує .gitignore), а у каталозі проєкту не з’являються output-файли: кожен тест працює у власному тимчасовому каталозі (tmp_path), а fixture output_dir додатково перевіряє відсутність .tmp-файлів. Тести не залежать від попереднього запуску.

12.7. Експеримент 7. Failure path

Для сервісу із зовнішніми залежностями перевірено і успішний, і невдалий сценарії; для невдалого перевіряється не лише exception, а й відсутність небажаних взаємодій.

Сценарій	Налаштування	Що перевіряється	Test
Усе успішно	get → None, send → ok	True, add і send викликано по одному разу з правильними аргументами	test_register_stores_student_and_sends_notification
Дублікат	get → існуючий студент	ValidationError, add та send не викликались	test_registering_duplicate_is_rejected_without_side_effects
Збій сповіщення	send.side_effect = NotificationError	False; студент усе одно збережений	test_failed_notification_does_not_undo_registration
Збій репозиторію	add.side_effect = RuntimeError	виняток піднято; send.assert_not_called()	test_repository_failure_is_propagated_and_nobody_is_notified
Збій async-залежності	get.side_effect = ConnectionError	виняток піднято	test_connection_error_of_dependency_is_propagated

13. Git

```
git init
git add .gitignore pyproject.toml config.yaml data src/student_app/__init__.py
git commit -m "Create project structure"
git add src/student_app/exceptions.py src/student_app/models.py
src/student_app/protocols.py src/student_app/repositories.py
src/student_app/notifiers.py src/student_app/services.py
git commit -m "Add domain model and business logic"
git add src/student_app/importer.py src/student_app/exporters.py
src/student_app/config.py src/student_app/reports.py src/student_app/pipeline.py
src/student_app/async_service.py src/student_app/main.py
git commit -m "Add import, export, configuration, reports and async service"
git add tests
git commit -m "Add unit and integration tests"
git add experiments README.md
git commit -m "Add experiments and documentation"
```

```
(.venv) PS C:\Users\Павло\OneDrive\Desktop\professional-python-lab6> git log --oneline
6431efe (HEAD -> main, origin/main) Add experiments and documentation
b90fcb7 Add unit and integration tests
1b0e92a Add import, export, configuration, reports and async service
86aa6ed Add domain model and business logic
5d27b40 Create project structure
```

Рис. 15. Результат git log --oneline

Репозиторій опубліковано на GitHub: https://github.com/Pavlo12112/professional-python-lab6

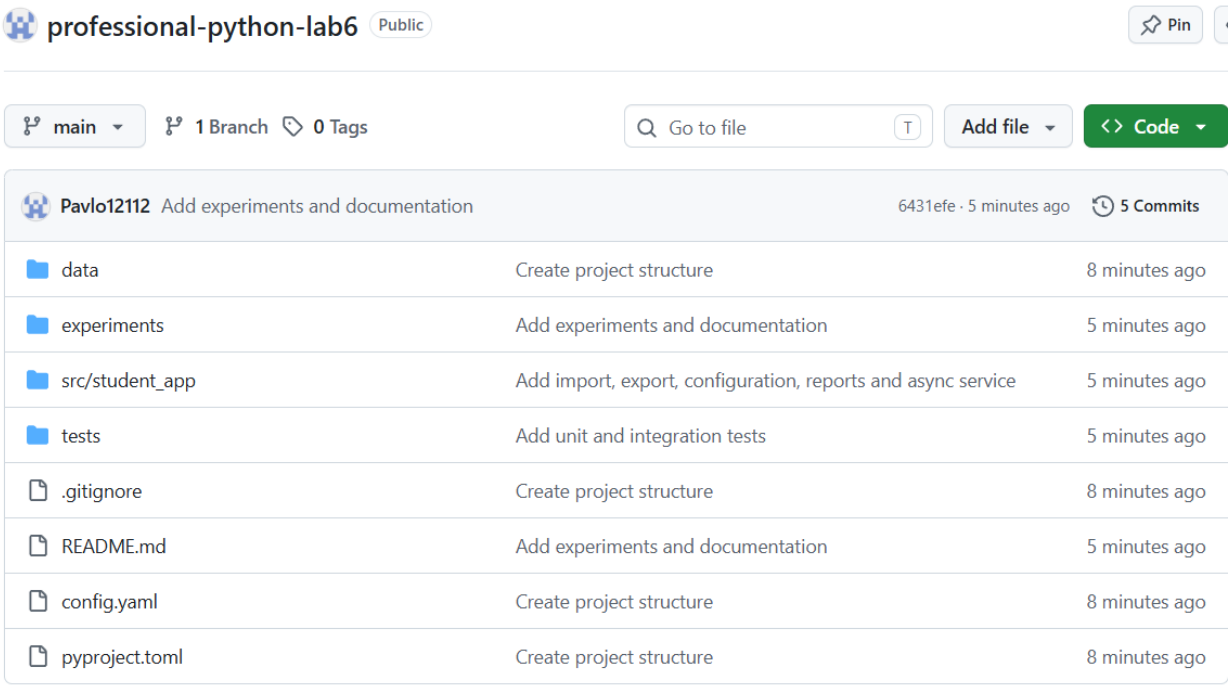


Рис. 16. Репозиторій на GitHub

14. Відповіді на контрольні запитання

1. Що таке automated test?

Програма, яка автоматично перевіряє поведінку іншої програми: викликає код із відомими даними та порівнює результат з очікуваним без участі людини.

2. Які основні переваги автоматизованого тестування?

Виявлення дефектів, захист від regressions, безпечніший refactoring, документування очікуваної поведінки, перевірка edge cases і підтримка якості в CI.

3. Що таке regression?

Помилка, яка з’являється у вже працюючій функціональності після зміни коду.

4. Що таке unit test?

Тест, що перевіряє невелику одиницю поведінки (функцію, клас) ізольовано від решти системи.

5. Що таке integration test?

Тест, що перевіряє взаємодію кількох компонентів разом (наприклад, CSV → валідація → JSON).

6. Чим integration test відрізняється від unit test?

Unit test перевіряє один компонент, зазвичай із підмінениими залежностями й дуже швидкий; integration test охоплює кілька компонентів і реальні ресурси (файли), тому повільніший, а локалізувати збій складніше.

7. Що показує test pyramid?

Рекомендоване співвідношення тестів: багато швидких unit tests внизу, менше integration tests посередині і мало дорогих end-to-end тестів нагорі.

8. Чому unit tests зазвичай більше?

Вони швидкі, дешеві й точно вказують на місце збою, тому ними можна покрити багато сценаріїв.

9. Що таке pytest?

Фреймворк для тестування Python із простим assert, fixtures, параметризацією та плагінами.

10. Як pytest знаходить tests?

За іменами: файли test_*.py або *_test.py, у них функції test_* (і класи Test*).

11. Якою командою запускається pytest?

Командою pytest у корені проєкту (або python -m pytest).

12. Для чого pytest -v?

Докладний вивід: для кожного тесту виводиться повне ім’я та результат.

13. Для чого pytest -x?

Зупиняє запуск після першого failure.

14. Що означає Arrange–Act–Assert?

Структура тесту: підготувати дані (Arrange), виконати дію (Act), перевірити результат (Assert).

15. Для чого використовується assert?

Для перевірки очікуваного результату; у pytest це звичайний assert, а при помилці pytest показує різницю значень.

16. Що таке assertion failure?

Стан, коли умова assert хибна: тест позначається як FAILED із поясненням, що саме відрізняється.

17. Чому floats не завжди слід порівнювати через ==?

Дійсні числа зберігаються з похибкою (наприклад, 0.1 + 0.2 ≠ 0.3), тому точне порівняння може давати хибний результат.

18. Для чого pytest.approx?

Порівнює числа з допустимою похибкою: assert result == pytest.approx(0.3).

19. Для чого pytest.raises?

Перевіряє, що код піднімає очікуваний виняток; якщо виняток не виник, тест падає.

20. Як перевірити exception message?

Параметром match (регулярний вираз): pytest.raises(ValueError, match="positive").

21. Як отримати exception object через exc_info?

with pytest.raises(...) as exc_info:, після блоку доступний exc_info.value із полями (наприклад, field, line_number).

22. Що таке boundary testing?

Перевірка значень на межах допустимого діапазону та поруч із ними, де помилки трапляються найчастіше.

23. Які boundaries потрібно тестувати для діапазону 0–100?

-0.01, 0, 0.01, середнє значення, 99.99, 100, 100.01.

24. Що таке parametrization?

Запуск одного тесту з різними наборами вхідних даних без дублювання коду.

25. Для чого pytest.mark.parametrize?

Описує імена параметрів і набори значень; для кожного набору pytest створює окремий test case.

26. Що дають ids параметризованих tests?

Зрозумілі імена в виводі (test_grade[minimum]), тому легко побачити, який саме випадок упав.

27. Що таке fixture?

Функція з @pytest.fixture, яка готує дані чи ресурс для тесту й передається йому за ім’ям параметра.

28. Для чого використовуються fixtures?

Для повторно використовуваного setup (об’єкти, тимчасові файли, mocks) і cleanup.

29. Що означає fixture setup?

Підготовка ресурсу перед тестом (код до return/yield).

30. Що означає fixture teardown?

Очищення після тесту (код після yield), що виконується навіть при падінні тесту.

31. Для чого yield у fixture?

Розділяє setup і teardown: значення після yield передається тесту, а код після нього виконується в кінці.

32. Які fixture scopes підтримує pytest?

function, class, module, package, session.

33. Який scope є default?

function: fixture створюється заново для кожного тесту.

34. Для чого conftest.py?

Для спільних fixtures і налаштувань, доступних усім тестам у каталозі та підкаталогах.

35. Чому fixtures із conftest.py не імпортують вручну?

pytest автоматично знаходить conftest.py і підставляє fixtures за іменем параметра; ручний імпорт порушує цей механізм.

36. Що таке tmp_path?

Вбудована fixture, що повертає Path до унікального тимчасового каталогу для тесту.

37. Чому tmp_path корисний?

Тести не пишуть у реальні каталоги проєкту, не конфліктують між собою й не залишають сміття.

38. Що таке test isolation?

Незалежність тестів: результат не залежить від порядку запуску, спільного стану чи залишків попередніх запусків.

39. Для чого monkeypatch?

Для тимчасової зміни змінних середовища, атрибутів, функцій, словників і поточного каталогу з автоматичним відкатом.

40. Як monkeypatch змінює environment variable?

monkeypatch.setenv("NAME", "value").

41. Як видалити environment variable у test?

monkeypatch.delenv("NAME", raising=False); raising=False запобігає помилці, якщо змінної немає.

42. Як підмінити function через monkeypatch?

monkeypatch.setattr(module, "function_name", fake_function), наприклад підміна reports.utcnow фіксованим часом.

43. Що таке test double?

Будь-який об’єкт, що замінює справжню залежність у тесті (Dummy, Stub, Fake, Mock, Spy).

44. Що таке Dummy?

Об’єкт-заглушка, який лише заповнює параметр і не використовується.

45. Що таке Stub?

Об’єкт, що повертає наперед задані відповіді (Mock з return_value).

46. Що таке Fake?

Спрощена робоча реалізація залежності (InMemoryStudentRepository замість бази даних).

47. Що таке Mock?

Об’єкт, що запам’ятовує виклики й дозволяє перевірити взаємодію (assert_called_once_with).

48. Що таке Spy?

Об’єкт, що записує, як його викликали, але зберігає справжню поведінку.

49. Для чого Mock?

Щоб ізолювати компонент від залежностей, задавати їхні відповіді й помилки та перевіряти взаємодію.

50. Що таке spec?

Параметр Mock(spec=Class), який обмежує mock методами справжнього класу, тому звернення до неіснуючого методу дає помилку.

51. Для чого return_value?

Задає значення, яке повертає виклик mock-методу.

52. Для чого side_effect?

Змушує mock піднімати виняток або повертати послідовність значень (моделювання збоїв).

53. Як перевірити, що method викликано один раз?

mock.method.assert_called_once() (або assert_called_once_with(...)).

54. Як перевірити parameters виклику?

mock.method.assert_called_once_with(arg1, arg2) або assert_called_with для останнього виклику.

55. Як перевірити, що method не викликався?

mock.method.assert_not_called().

56. Що таке state testing?

Перевірка кінцевого стану (результату, статусу, вмісту сховища) після дії.

57. Що таке interaction testing?

Перевірка того, як компонент взаємодіяв із залежностями (які методи і з якими аргументами викликав).

58. Коли interaction testing виправдане?

Коли взаємодія сама є результатом: сповіщення надіслано або не надіслано, платіж не виконано після помилки.

59. Що таке patch?

unittest.mock.patch тимчасово замінює об’єкт на mock (як декоратор або context manager).

60. Чому patch потрібно виконувати там, де name використовується?

Бо модуль, що імпортував name, тримає власне посилання на нього; підміна в місці визначення на це посилання не вплине.

61. Що таке AsyncMock?

Mock для асинхронних залежностей: його виклик повертає awaitable, а return_value і side_effect працюють при await.

62. Як перевірити awaited call?

assert_awaited_once(), assert_awaited_once_with(...), assert_awaited().

63. Що таке flaky test?

Тест, який без зміни коду то проходить, то падає (через випадковість, час, мережу чи порядок запуску).

64. Чому flaky tests небезпечні?

Вони знецінюють результати suite, приховують реальні дефекти й змушують ігнорувати падіння.

65. Що таке deterministic test?

Тест, який при однакових вхідних даних завжди дає однаковий результат.

66. Чому unit test не повинен залежати від Internet?

Інтернет повільний і нестабільний: тест стає flaky та непридатним для швидкого й повторюваного запуску.

67. Коли в integration test доцільний real filesystem?

Коли перевіряється саме взаємодія з файлами (CSV, JSON, YAML, atomic write); використовують тимчасові каталоги через tmp_path.

68. Чому не потрібно mock-ати все?

Тоді тест перевіряє лише mocks, а не реальну поведінку, і не виявить помилок у взаємодії компонентів.

69. Що таке over-mocking?

Надмірне використання mocks, коли підмінено майже все, і тест залежить від реалізації, а не від результату.

70. Що таке coverage?

Показник того, яка частина коду була виконана під час тестів.

71. Що таке statement coverage?

Відсоток виконаних рядків (операторів) коду.

72. Що таке branch coverage?

Відсоток виконаних гілок умов (і True, і False для кожного if).

73. Чим вони відрізняються?

Statement coverage може бути 100%, навіть якщо гілка else жодного разу не виконувалась; branch coverage вимагає перевірити обидві гілки.

74. Як отримати coverage через pytest-cov?

pytest --cov=student_app (з --cov-branch для branch coverage).

75. Для чого --cov-report=term-missing?

Показує в терміналі номери рядків, які тести не виконали.

76. Для чого HTML coverage report?

Дає зручний перегляд файлів із підсвіткою непокритих рядків і гілок у браузері (htmlcov/index.html).

77. Чи гарантує 100% coverage відсутність bugs?

Ні: код може виконуватись без перевірки результату, а деякі помилки (логічні, у відсутніх сценаріях) взагалі не залежать від покриття.

78. Чому meaningful assertions важливіші за coverage percentage?

Coverage показує лише виконання коду, а саме assertions перевіряють коректність результату (експерименти 1 і 2).

79. Що таке coverage threshold?

Мінімальний допустимий відсоток покриття (fail_under = 80); при нижчому значенні запуск завершується помилкою.

80. Чому tests повинні бути незалежними один від одного?

Щоб порядок запуску не впливав на результат, тести можна було запускати окремо, а збій одного не ламав інші.

15. Висновок

У лабораторній роботі створено повноцінний automated test suite для проєкту обліку студентів (варіант 1): 82 test functions, які після параметризації дають 113 test cases (103 unit та 10 integration). Використано fixtures (у тому числі fixture factory, fixture з yield і autouse), параметризацію з ids, pytest.raises із перевіркою повідомлення та полів exception, pytest.approx, tmp_path, monkeypatch (змінні середовища, підміна функції, детермінований час, збій запису файла), Mock із spec, return_value, side_effect і interaction assertions, autospec та AsyncMock.

Експерименти показали, що слабкий тест виконує код, не перевіряючи результат, а 100% coverage не гарантує правильності; branch coverage виявляє пропущену гілку, яку statement coverage ховає; flaky tests непередбачувані й небезпечні; Mock добре підходить для перевірки взаємодії та збоїв, а fake — для перевірки стану. Навмисний дефект у calculate_average виявили вісім тестів різних рівнів, а негативні сценарії (збій залежностей без небажаних

взаємодій) покрито окремими тестами. Test suite запускається однією командою, формує звіт про statement та branch coverage і має поріг 80%.